

JAVA CHEAT SHEET — TESD 1400 CORE

BASIC STRUCTURE OF A JAVA PROGRAM

```
public class ClassName {  
    public static void main(String[] args) {  
        // your code here  
    }  
}
```

Notes:

- File name must match class name (ClassName.java).
- main() is where the program starts running.

PRINTING OUTPUT

```
System.out.print("text"); // prints, no new line  
System.out.println("text"); // prints and moves to next line  
System.out.println(5 + 3); // you can print numbers and math
```

COMMENTS

Single-line comment:

```
// this is a comment
```

Multi-line comment:

```
/* this is  
a multi-line comment */
```

VARIABLES + DATA TYPES

```
int age = 19; // whole number  
double price = 3.99; // decimal number  
boolean done = true; // true or false  
char grade = 'A'; // single character  
String name = "Jeff"; // text (String is capitalized)
```

Final / constant:

```
final double TAX = 0.07; // cannot change later
```

BASIC MATH OPERATORS

- add
- subtract
- multiply
/ divide
% remainder (modulo)

Examples:

```
int x = 10 % 3; // x = 1
```

```
double y = 7 / 2; // y = 3.0 if both are ints first (do 7.0/2 for 3.5)
```

INPUT FROM USER (SCANNER)

```
import java.util.Scanner;
```

```
Scanner input = new Scanner(System.in);
```

```
int n = input.nextInt(); // read int
```

```
double d = input.nextDouble(); // read double
```

```
String s = input.next(); // one word
```

```
String line = input.nextLine(); // whole line of text
```

Example:

```
System.out.print("Enter age: ");
```

```
int age = input.nextInt();
```

STRING BASICS

```
String s = "hello";
```

```
s.length() // number of characters
```

```
s.toUpperCase() // "HELLO"
```

```
s.toLowerCase() // "hello"
```

```
s.charAt(0) // 'h'
```

```
s.equals("hello") // true/false, use this NOT ==
```

Don't compare strings with ==
Use .equals()

IF / ELSE (SELECTIONS)

```
if (condition) {  
    // do this  
} else if (anotherCondition) {  
    // do this  
} else {  
    // fallback  
}
```

Relational operators:

== equal
!= not equal

greater than
< less than
= greater or equal
<= less or equal

Logical operators:

&& and
|| or
! not

Example:

```
if (score >= 90) {  
    grade = 'A';  
} else if (score >= 80) {  
    grade = 'B';  
} else {  
    grade = 'C';  
}
```

SWITCH STATEMENT

```
switch (day) {  
    case 1:  
        System.out.println("Monday");  
        break;  
    case 2:  
        System.out.println("Tuesday");  
}
```

```
break;
default:
System.out.println("Unknown");
}
```

- break; is important, or it “falls through.”

WHILE LOOP

```
while (condition) {
// repeat this while condition is true
}
```

Example:

```
int count = 0;
while (count < 5) {
System.out.println("Hi");
count++;
}
```

FOR LOOP

```
for (int i = 0; i < 5; i++) {
System.out.println("Loop #" + i);
}
```

Pattern:

start ; condition ; update

DO-WHILE LOOP

```
do {
// body runs at least once
} while (condition);
```

Example:

```
int n;
do {
System.out.print("Enter positive number: ");
n = input.nextInt();
} while (n <= 0);
```

RANDOM NUMBERS

```
import java.util.Random;

Random r = new Random();
int x = r.nextInt(6); // 0 to 5
int die = r.nextInt(6) + 1; // 1 to 6

double y = Math.random(); // 0.0 <= y < 1.0
int pick = (int)(Math.random() * 10); // 0-9
```

METHODS (BASICS)

Define a method (function) outside main but inside the class:

```
public static int add(int a, int b) {
    return a + b;
}
```

Call it:

```
int sum = add(3, 4);
```

Void method (no return):

```
public static void sayHi() {
    System.out.println("Hi");
}
```

ARRAYS (1D)

```
int[] nums = new int[5]; // 5 spots, all 0
nums[0] = 42; // set first
int x = nums[0]; // read first
```

```
int[] grades = {90, 85, 70, 100}; // shortcut create
```

length:

```
grades.length // number of elements
```

loop over array:

```
for (int i = 0; i < grades.length; i++) {
    System.out.println(grades[i]);
}
```

enhanced for loop (for-each):

```
for (int g : grades) {  
    System.out.println(g);  
}
```

2D ARRAYS (BASICS)

```
int[][] grid = new int[3][3];  
grid[0][1] = 7;
```

Loop through 2D:

```
for (int row = 0; row < grid.length; row++) {  
    for (int col = 0; col < grid[row].length; col++) {  
        System.out.print(grid[row][col] + " ");  
    }  
    System.out.println();  
}
```

CASTING / TYPE CONVERSION

```
double d = 5; // int -> double is allowed  
int n = (int)3.7; // force cut to 3
```

```
int n2 = Integer.parseInt("42"); // "42" → 42  
double d2 = Double.parseDouble("2.5"); // "2.5" → 2.5
```

MATH CLASS QUICK HITS

```
Math.pow(a, b) // a^b  
Math.sqrt(x) // square root  
Math.abs(x) // absolute value  
Math.max(a, b) // bigger of two  
Math.min(a, b) // smaller of two  
Math.round(x) // rounds to nearest int
```

BOOLEAN LOGIC EXAMPLE

```
boolean ok = (age >= 18 && hasID);  
if (ok) {  
    System.out.println("You may enter.");  
} else {
```

```
System.out.println("Not allowed.");  
}
```

COMMON ERROR REMINDERS

- Use `.equals()` to compare Strings, not `==`
- Every `{` needs a matching `}`
- Arrays start at index 0, not 1
- Integer division drops decimals (`7 / 2 == 3`)
- Scanner `nextInt()` leaves the newline, so sometimes you need an extra `nextLine()` after it before reading text

MINI PATTERNS YOU REUSE A LOT

Print a menu and get choice:

```
System.out.println("1) Start\n2) Quit");  
int choice = input.nextInt();
```

Sum loop:

```
int sum = 0;  
for (int i = 0; i < nums.length; i++) {  
    sum += nums[i];  
}
```

Validate input:

```
while (num < 0) {  
    System.out.println("Enter again:");  
    num = input.nextInt();  
}
```

Method template:

```
public static returnType name(params) {  
    // work  
    return value;  
}
```